



7

22

2640 sp

نظري

كلية الهندسة المعلوماتية

السنة الثالثة

المزيد من لغة ال SQL

د. سعيد دسوقي

محتوى مجاني غير مخصص للبيع التجاري

RB Informatics; 05/06/2026

قواعد معطيات

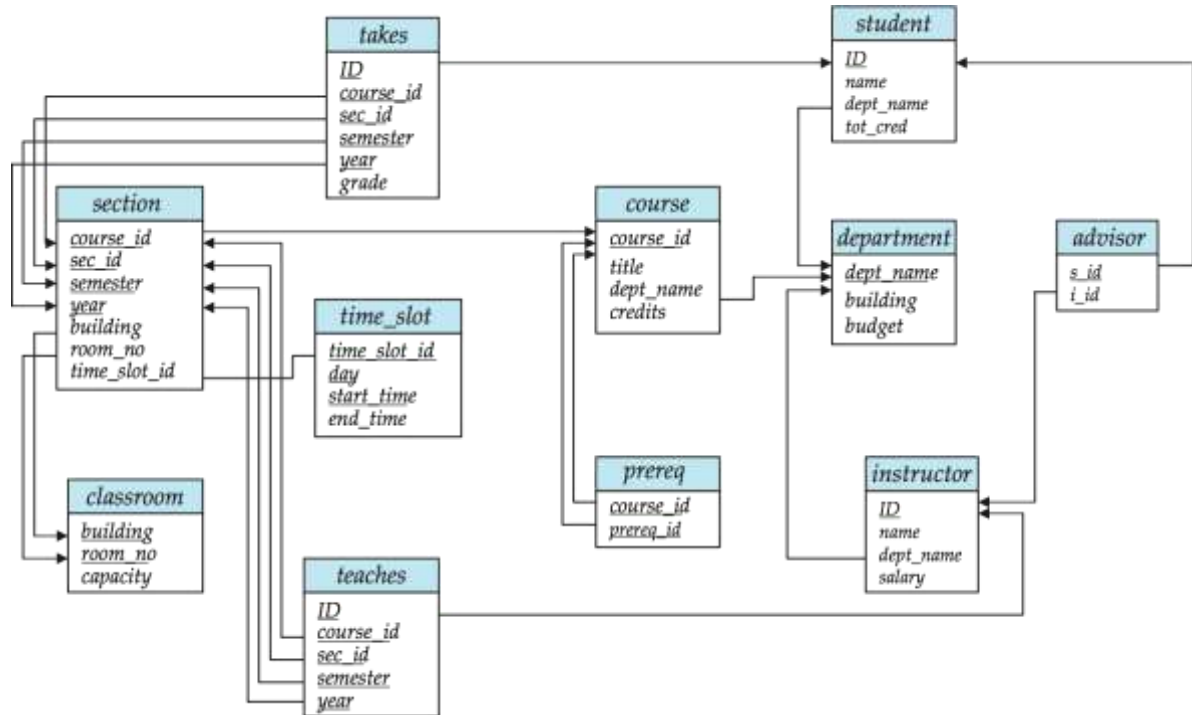
السلام عليكم ورحمة الله وبركاته

نتناول هذه المحاضرة مجموعة من المفاهيم المتقدمة في لغة SQL، مثل :

الضم الخارجي (Outer Join) ، والروى (Views) ، وعمليات تعديل البيانات ، والمداولات (Transactions) ،

إضافةً إلى بعض القيود وأنماط المعطيات وإدارة الصلاحيات، بهدف توسيع إمكانيات التعامل مع قواعد المعطيات العلائقية .

مخطط بنية قاعدة معطيات الجامعة :



يفشل المخطط السابق **بنية قاعدة المعطيات (Schema)** ، أي تصميم قاعدة البيانات الذي يحدد أسماء الجداول، وواصفاتها (Attributes)، والعلاقات والمفاتيح الأساسية دون أن يحتوي على البيانات الفعلية المخزنة.

أما الجداول التالية فتتمثل **النسخة** (Instance)، وهي البيانات الفعلية الموجودة داخل قاعدة المعطيات في لحظة معينة، مثل بيانات الأقسام، والمدرسين، والمواد الدراسية، وغيرها من السجلات.

ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

dept_name	building	budget
Biology	Watson	90000
Comp. Sci.	Taylor	100000
Elec. Eng.	Taylor	85000
Finance	Painter	120000
History	Painter	50000
Music	Packard	80000
Physics	Watson	70000

course_id	title	dept_name	credits
BIO-101	Intro. to Biology	Biology	4
BIO-301	Genetics	Biology	4
BIO-399	Computational Biology	Biology	3
CS-101	Intro. to Computer Science	Comp. Sci.	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3
CS-319	Image Processing	Comp. Sci.	3
CS-347	Database System Concepts	Comp. Sci.	3
EE-181	Intro. to Digital Systems	Elec. Eng.	3
FIN-201	Investment Banking	Finance	3
HIS-351	World History	History	3
MU-199	Music Video Production	Music	3
PHY-101	Physical Principles	Physics	4

course_id	prereq_id
BIO-301	BIO-101
BIO-399	BIO-101
CS-190	CS-101
CS-315	CS-101
CS-319	CS-101
CS-347	CS-101
EE-181	PHY-101

course_id	sec_id	semester	year	building	room_number	time_slot_id
BIO-101	1	Summer	2009	Painter	514	B
BIO-301	1	Summer	2010	Painter	514	A
CS-101	1	Fall	2009	Packard	101	H
CS-101	1	Spring	2010	Packard	101	F
CS-190	1	Spring	2009	Taylor	3128	E
CS-190	2	Spring	2009	Taylor	3128	A
CS-315	1	Spring	2010	Watson	120	D
CS-319	1	Spring	2010	Watson	100	B
CS-319	2	Spring	2010	Taylor	3128	C
CS-347	1	Fall	2009	Taylor	3128	A
EE-181	1	Spring	2009	Taylor	3128	C
FIN-201	1	Spring	2010	Packard	101	B
HIS-351	1	Spring	2010	Painter	514	C
MU-199	1	Spring	2010	Packard	101	D
PHY-101	1	Fall	2009	Watson	100	A



Outer Join الضم الخارجي :

في البداية نتذكر أن Natural Join يقوم بربط جدولين اعتماداً على الواصفة المشتركة (العمود المشترك) بينهما، بحيث يعرض فقط السجلات التي يوجد لها تطابق في كلا الجدولين.

مثال :

course_id	title	dept_name	credits	course_id	prereq_id
BIO-301	Genetics	Biology	4	BIO-301	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-190	CS-101
CS-315	Robotics	Comp. Sci.	3	CS-347	CS-101

مثال :

course natural join prereq

course_id	title	dept_name	credits	prere_id
BIO-301	Genetics	Biology	4	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-101

شرح : في المثال السابق تتم المطابقة باستخدام الواصفة course_id لأنها مشتركة بين جدولي course و prereq.

وبالتالي فإن أي سجل لا يجد قيمة مطابقة لـ course_id الخاص به في الجدول الآخر لن يظهر في نتيجة الـ Natural Join.

Left Outer Join

- في Natural Left Outer Join يتم الاحتفاظ بجميع سجلات الجدول الموجود على اليسار، حتى لو لم تجد قيمة مطابقة في الجدول الموجود على اليمين.
- وفي حال عدم وجود تطابق، توضع القيمة NULL في الحقول التابعة للجدول الأيمن.

مثال :

course natural left outer join prereq

course_id	title	dept_name	credits	prere_id
BIO-301	Genetics	Biology	4	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-101
CS-315	Robotics	Comp. Sci.	3	null

Right Outer Join

- في Natural Right Outer Join يتم الاحتفاظ بجميع سجلات الجدول الموجود على اليمين، حتى لو لم تجد قيمة مطابقة في الجدول الموجود على اليسار.
- وعند عدم وجود تطابق، تظهر قيم NULL مكان الحقول المفقودة من الجدول الأيسر.

مثال :

course **natural right outer join** prereq

course_id	title	dept_name	credits	prere_id
BIO-301	Genetics	Biology	4	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-101
CS-347	null	null	null	CS-101

Full Outer Join

- أما Natural Full Outer Join فيجمع بين الحالتين السابقتين، حيث يحتفظ بجميع سجلات الجدولين معاً، سواء وجدت لها مطابقة أم لا، وتُمثل القيم غير الموجودة باستخدام NULL

مثال :

course **natural full outer join** prereq

course_id	title	dept_name	credits	prere_id
BIO-301	Genetics	Biology	4	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-101
CS-315	Robotics	Comp. Sci.	3	null
CS-347	null	null	null	CS-101

ملاحظة

يبقى اسم العملية Natural Join موجوداً في جميع الحالات السابقة، لأن الربط يتم تلقائياً اعتماداً على الواصفة المشتركة التي تحمل الاسم نفسه في الجدولين.

- كيف نحدد أي الجدولين هو اليسار وأيها اليمين؟

يتم تحديد ذلك من ترتيب الجداول داخل الاستعلام نفسه.

فمثلاً: course NATURAL LEFT OUTER JOIN prereq

الجدول course هو الجدول الأيسر (Left)، وprereq هو الجدول الأيمن (Right).

Views الرؤى :

- تُعد الرؤية (View) جدولاً افتراضياً (Virtual Table) ، يُستخدم لتسهيل التعامل مع الاستعلامات المعقدة وتبسيطها، كما يمكن استخدامه لإخفاء بعض البيانات عن بعض المستخدمين من خلال عرض جزء محدد من الجدول الأصلي، أي كأنه استعلام له اسم.
- وسميت View لأنها تمثل نافذة (View) على قاعدة البيانات، حيث تحدد مجموعة معينة من الصفوف والأعمدة باستخدام تعليمات SELECT و WHERE دون الحاجة إلى إنشاء جدول جديد.
- لا يتم تخزين بيانات الرؤية فعلياً داخل قاعدة البيانات، وإنما يتم تخزين صيغة الاستعلام (Query) فقط، وعند استدعاء الرؤية يتم تنفيذ هذا الاستعلام على الجداول الأصلية لإحضار البيانات المطلوبة، لذلك تسمى جدولاً افتراضياً.

- يتم إنشاء الرؤية باستخدام التعليمة:

```
create view view_name as <query expression>
```

أي يتم إعطاء اسم للرؤية وربطها باستعلام SQL معين.

- بعد إنشاء الرؤية يمكن استخدامها تماماً كما لو كانت جدولاً عادياً في أوامر SELECT وغيرها من الاستعلامات.
- كما يمكن أيضاً إنشاء رؤية جديدة اعتماداً على رؤية موجودة مسبقاً

مثال:

```
create view faculty as
select ID, name, dept_name
from instructor;
```

تقوم هذه الرؤية بعرض رقم المدرس واسمه والقسم الذي ينتمي إليه فقط وإخفاء الراتب.

- بعد ذلك يمكن استخدامها كما يلي:

```
select name
from faculty
where dept_name = 'Biology';
```

- كما يمكن إنشاء رؤية أخرى اعتماداً على الرؤية السابقة:

```
create view bio_faculty as
select *
from faculty
where dept_name = 'Biology';
```

بحيث تحتوي هذه الرؤية على مدرسي قسم الأحياء فقط، مع إمكانية استخدامها لاحقاً كما لو كانت جدولاً مستقلاً.

التعديل على قاعدة المعطيات

ملاحظة : قبل أن نبدأ بعمليات تعديل البيانات، نشير إلى أن تعليمة SELECT لا تقوم بتعديل البيانات المخزنة داخل قاعدة المعطيات إطلاقاً، حتى في حال احتوت على عمليات حسابية أو تعابير رياضية، وإنما يقتصر دورها على عرض النتائج المطلوبة فقط دون التأثير على البيانات الأصلية.

الإدخال INSERT :

- تُستخدم تعليمة INSERT لإضافة سجلات (صفوف) جديدة إلى أحد جداول قاعدة المعطيات. ويمكن إدخال القيم مباشرة، أو إدخال نتائج استعلام SELECT داخل الجدول.

المثال الأول:

```
insert into course  
values ('CS-437', 'Database Systems', 'Comp. Sci.', 4);
```

- الشرح : إذا لم نذكر أسماء الأعمدة في تعليمة INSERT، فيجب إدخال القيم بنفس ترتيب الأعمدة كما هي معرفة في الجدول عند إنشائه (CREATE TABLE).

المثال الثاني:

```
insert into course (course_id, title, credits, dept_name)  
values ('CS-437', 'Database Systems', 4, 'Comp. Sci.');
```

- إذا ذكرنا أسماء الأعمدة، فيتم إدخال القيم بنفس الترتيب الذي كتبنا به أسماء الأعمدة، ولا يشترط ترتيب الجدول الأصلي.

ملاحظة :

أثناء عملية الإدخال يتم التحقق من جميع القيود (Constraints) ، مثل:

1. عدم تكرار قيمة Primary Key.
 2. وجود قيمة Foreign Key في الجدول المرجعي.
 3. عدم إدخال NULL في الحقول المعرفة بـ NOT NULL.
 4. توافق نوع البيانات مع نوع العمود (رقم، نص، تاريخ، ...).
 5. التحقق من أي قيود أخرى مثل CHECK أو UNIQUE.
- وإذا خالف الإدخال أي قيد من هذه القيود فسيتم رفض التعليمة.

المثال الثالث:

```
insert into student  
select ID, name, dept_name, 0  
from instructor;
```

- يقوم هذا الاستعلام بنسخ بيانات جميع المدرسين وإدخالها في جدول student، حيث يتم أخذ:
ID - name - dept_name ، من جدول instructor ، بينما يتم وضع القيمة 0 مباشرة في الحقل tot_cred لجميع السجلات الجديدة.
- أي أن نتيجة الـ SELECT تستخدم مباشرة كبيانات لإضافتها إلى الجدول.

المثال الرابع:

```
insert into table1  
select * from table1;
```

- يقوم هذا الاستعلام بنسخ جميع صفوف الجدول وإعادة إدخالها مرة أخرى داخل الجدول نفسه.
- إذا كان الجدول يحتوي على Primary Key فستفشل عملية الإدخال ويصدر خطأ، لأن القيم المكررة ستخالف شرط عدم تكرار المفتاح الأساسي.
- أما إذا لم يكن هناك Primary Key أو أي قيد يمنع التكرار، فسيتم إدخال جميع الصفوف مرة أخرى، مما يؤدي إلى تكرار البيانات داخل الجدول.

التعديل UPDATE

- تُستخدم تعليمة UPDATE لتعديل القيم الموجودة مسبقاً داخل الجدول، سواء كان التعديل على سجل واحد أو عدة سجلات، وذلك وفق شرط يحدد بواسطة WHERE .
- وإذا لم يُذكر شرط، فسيتم تعديل جميع السجلات الموجودة في الجدول.

المثال الأول:

```
update instructor  
set salary = salary * 1.03  
where salary > 100000;
```

- يقوم هذا الاستعلام بزيادة رواتب المدرسين الذين تزيد رواتبهم عن 100000 بنسبة 3% فقط، بينما تبقى بقية الرواتب دون تعديل.

المثال الثاني:

```
update instructor  
set salary = salary * 1.05  
where salary <= 100000;
```

■ يقوم هذا الاستعلام بزيادة رواتب المدرسين الذين تقل أو تساوي رواتبهم 100000 بنسبة 5%.

ملاحظة مهمة

- إذا قمنا بتنفيذ المثالين السابقين بترتيب غير مناسب، فقد يحدث خطأ منطقي، لأن بعض الرواتب قد تتغير في التعليم الأولى ثم تحقق شرط التعليم الثانية، أو العكس، وبالتالي يحصل السجل نفسه على تعديلين متتاليين.
- ولتجنب هذه المشكلة يتم استخدام تعليمة CASE بحيث يتم اتخاذ القرار لكل سجل مرة واحدة فقط.

المثال الثالث:

```
update instructor  
set salary = case  
    when salary <= 100000 then salary * 1.05  
    else salary * 1.03  
end
```

- تسمح تعليمة CASE بتنفيذ جميع الحالات ضمن استعلام واحد، بحيث يتم اختيار الزيادة المناسبة لكل مدرس مرة واحدة فقط، مما يمنع حدوث أي تأثير مزدوج على الرواتب ويضمن صحة النتائج.



المثال الرابع:

```

update student S
set tot_cred =
(
  select sum(credits)
  from takes natural join course
  where S.ID = takes.ID
  and takes.grade <> 'F'
  and takes.grade is not null
);

```

- يقوم هذا الاستعلام بحساب مجموع الساعات المعتمدة (Credits) لكل طالب محقق للشروط، ثم تخزينها في الحقل tot_cred داخل جدول student.
- ويتم حساب المجموع من خلال الربط بين جدولي takes و course، ثم جمع قيم credits الخاصة بالمقررات التي نجح بها الطالب.
- يشترط في عملية الجمع:
 - 1- أن تكون العلامة لا تساوي F حتى لا تحسب المواد الراسبة.
 - 2- وأن تكون العلامة ليست NULL للتأكد من وجود علامة فعلية للمقرر، أي أن الطالب أنهى المقرر وتم تسجيل نتيجته.
- ويسمى الاستعلام السابق بـ الاستعلام المترابط (Correlated Query).

الاستعلام المترابط (Correlated Query)

- يسمى هذا النوع من الاستعلامات Correlated Query لأنه يعتمد على الجدول الخارجي أثناء تنفيذ الاستعلام الداخلي، ويظهر ذلك في الشرط:

where S.ID = takes.ID

 حيث تعتمد قيمة S.ID على السجل الحالي من الجدول student، لذلك يعاد تنفيذ الاستعلام الداخلي لكل طالب على حدة.
- أما Subquery العادية فتنفذ مرة واحدة فقط، ثم تستخدم نتيجتها داخل الاستعلام الخارجي.
- ولهذا السبب يكون تنفيذ Correlated Query أكثر كلفة من Subquery العادية، لأن الاستعلام الداخلي يتكرر مع كل سجل من سجلات الجدول الخارجي (التعقيد $N*N$).

تذكرة بـSubquery من المحاضرة السابقة:

```
select distinct course_id
from section
where semester = 'Fall' and year= 2009 and
      course_id in (select course_id
                    from section
                    where semester = 'Spring' and year= 2010);
```

نلاحظ أن الاستعلام الداخلي يتنفذ مرة واحدة، ويقوم الاستعلام الخارجي بالمقارنة بنتيجته عند كل سطر من مواد خريف 2009.

ملاحظة

الرمز: <> يعني لا يساوي (Not Equal) في لغة SQL ،
وبالتالي: takes.grade <> 'F'
تعني أن علامة الطالب ليست F أي أنه غير راسباً في ذلك المقرر .

الحذف DELETE

- تُستخدم تعليمة DELETE لحذف السجلات (الصفوف) من أحد جداول قاعدة المعطيات، ويمكن حذف جميع السجلات أو جزء منها وفق شرط محدد باستخدام WHERE .
- وتجدر الإشارة إلى أن DELETE تحذف البيانات الموجودة داخل الجدول فقط، ولا تحذف الجدول نفسه أو بنيته.

المثال الأول:

```
delete from instructor;
```

- يقوم هذا الاستعلام بحذف جميع السجلات الموجودة في جدول instructor ، بينما يبقى الجدول فارغاً موجوداً داخل قاعدة المعطيات ويمكن استخدامه لاحقاً لإضافة بيانات جديدة.

المثال الثاني:

```
delete from instructor
where dept_name = 'Finance';
```

- يقوم هذا الاستعلام بحذف جميع المدرسين الذين ينتمون إلى قسم Finance فقط، بينما تبقى بقية السجلات دون تغيير.

المثال الثالث:

```
delete from instructor
where dept_name in
(
  select dept_name
  from department
  where building = 'Watson'
);
```

- في هذا المثال لم يتم استخدام اسم المبنى مباشرة في جدول instructor، لأن هذا الجدول لا يحتوي على الوصفة building.
- لذلك يتم أولاً تنفيذ الاستعلام الفرعي (Subquery) وهو:

```
select dept_name
from department
where building = 'Watson'
```

- ويقوم هذا الاستعلام بإحضار أسماء جميع الأقسام الموجودة في المبنى Watson.
- بعد ذلك تستخدم تعليمة IN هذه النتائج لحذف جميع المدرسين الذين ينتمون إلى تلك الأقسام.
- أي أن التنفيذ يتم على مرحلتين:
 - 1- استخراج أسماء الأقسام الموجودة في المبنى Watson.
 - 2- حذف جميع المدرسين الذين تنتمي قيمة dept_name لديهم إلى هذه المجموعة.

المثال الرابع:

```
delete from instructor
where salary <
(
  select avg(salary)
  from instructor
);
```

- يقوم الاستعلام الفرعي أولاً بحساب متوسط رواتب جميع المدرسين باستخدام الدالة AVG.
- بعد ذلك يقوم الاستعلام الخارجي بحذف جميع المدرسين الذين تكون رواتبهم أقل من متوسط الرواتب المحسوب.
- وبذلك يتم حذف جميع السجلات التي تقل رواتبها عن المتوسط العام لرواتب المدرسين.

التعديل على الرؤى (Views)

- بما أن الرؤية (View) هي جدول افتراضي مشتق من استعلام على الجداول الأصلية، فإن أي تعديل يتم عليها يجب أن يكون قابلاً للترجمة إلى تعديل مكافئ على الجداول الأساسية.
- فإذا أمكن تحويل التعديل بشكل واضح إلى الجداول الأصلية نُفذ التعديل، أما إذا لم يكن ذلك ممكناً فيتم رفض العملية.

المثال الأول:

```
create view faculty as
select ID, name, dept_name
from instructor;
```

■ ثم:

```
insert into faculty
values ('30765', 'Green', 'Music');
```

- في هذا المثال تعتمد الرؤية على جدول واحد فقط وهو instructor، لذلك من الممكن إضافة سجل جديد إليها.

المثال الثاني:

```
create view instructor_info as
select ID, name, building
from instructor, department
where instructor.dept_name = department.dept_name;
```

■ ثم:

```
insert into instructor_info
values ('69987', 'White', 'Taylor');
```

- في هذا المثال أصبحت الرؤية مبنية على جدولين هما instructor و department.
- لذلك عند محاولة الإدخال لا يستطيع النظام معرفة إلى أي جدول يجب إضافة البيانات، بالإضافة إلى أنه لا يملك dept_name للأستاذ المراد إضافة الاسم والرقم والبناء الخاص به، وبالتالي لا يمكن ترجمة عملية الإدخال، ولهذا يتم رفضها.

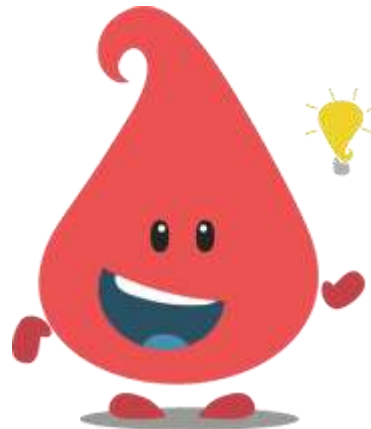
- التعديل على View ينعكس مباشرة على الجداول الأصلية ، لأنه لا توجد بيانات مخزنة داخل الرؤية نفسها.
- بعض عمليات التعديل لا يمكن ترجمتها إلى الجداول الأصلية، لذلك يتم رفضها.
- بعض عمليات التعديل لا يمكن ترجمتها نهائياً، خاصة إذا كانت الرؤية مبنية على أكثر من جدول أو تحتوي على عمليات معقدة.
- بعض أنظمة إدارة قواعد المعطيات (DBMS Engines) تمنع التعديل على الرؤى نهائياً، بينما تسمح أنظمة أخرى بذلك فقط إذا كانت الرؤية بسيطة (Simple View).

وتعتبر الرؤية بسيطة إذا تحققت الشروط التالية:

1. أن تعتمد على جدول واحد فقط في الجزء FROM .
2. ألا تحتوي على توابع تجميعية (Aggregate Functions) .
3. ألا تحتوي على DISTINCT .
4. أن يكون بالإمكان إعطاء قيمة NULL لأي عمود غير ظاهر في الرؤية عند الحاجة.
5. ألا تحتوي على تعليمات GROUP BY أو HAVING .

لذلك:

فإن إمكانية التعديل على الرؤية تعتمد على مدى قدرة النظام على تحويل هذا التعديل إلى تعديل صحيح وواضح على الجداول الأصلية.



مفهوم المداولة (Transaction)

- Transaction هي مجموعة من العمليات تُعامل كوحدة عمل واحدة غير قابلة للتجزئة (Atomic Unit)، بحيث إما أن تنفذ جميع تعليماتها بنجاح أو لا يُنفذ أي جزء منها.
- وتستخدم لتنظيم العمل المتزامن (التنافسي) بين عدة مستخدمين، وضمان سلامة البيانات عند تنفيذ عمليات التعديل على قاعدة المعطيات.

COMMIT و ROLLBACK :

- تبدأ Transaction من آخر عملية COMMIT أو من بداية الجلسة إذا لم يوجد Commit سابق ، وتنتهي عند تنفيذ COMMIT جديد أو ROLLBACK .
- COMMIT - : يقوم بتثبيت جميع التعديلات بشكل نهائي داخل قاعدة البيانات.
- ROLLBACK - : يقوم بإلغاء جميع التعديلات التي تمت منذ آخر COMMIT وإعادة قاعدة البيانات إلى حالتها السابقة.
- أي أن Transaction هي جميع العمليات الواقعة بين عمليتي تثبيت متتاليتين.

- في البرمجة يمكن تحديد حدود المداولة (Transaction) بشكل صريح، وذلك باستخدام:

```
BEGIN ATOMIC
...
END
```

- حيث تُعتبر جميع التعليمات الموجودة بين BEGIN ATOMIC و END واحدة (Transaction) تُنفذ كوحدة عمل واحدة، بحيث يتم تثبيت جميع التعديلات معاً أو التراجع عنها معاً عند الحاجة.

مثال

- إذا قمنا بتعديل رواتب المدرسين بزيادة 5%، فإن المستخدم الذي نفذ التعديل سيرى القيم الجديدة داخل جلسته.
- أما المستخدمون الآخرون فلن يروا هذه التعديلات إلا بعد تنفيذ: COMMIT;
- لأن التعديلات ما تزال ضمن ال Transaction ولم تثبت نهائياً بعد.
- ولهذا يحتفظ النظام بنسخة من الحالة السابقة حتى يستطيع تنفيذ ROLLBACK عند الحاجة قبل عملية التثبيت.

مثال اربيسيزي على ROLLBACK :

```
update instructor
set salary = salary * 1.05;
rollback;
```

بعد تنفيذ ROLLBACK تعود جميع الرواتب إلى قيمها الأصلية، وكأن عملية التعديل لم تنفذ أساساً.

ملاحظات

- أثناء ال Transaction يحتفظ النظام بالتعديلات مؤقتاً حتى يتم تثبيتها أو إلغاؤها.
- إن حدوث Error أثناء تنفيذ إحدى التعليمات لا يؤدي تلقائياً إلى تنفيذ ROLLBACK. فعملية ROLLBACK تُستخدم للتراجع عن التعديلات عند طلبها صراحةً أو عند حدوث فشل في تنفيذ المعاملة (Transaction) مثل : تعطل النظام أو انقطاع الكهرباء أو انهيار محرك قاعدة البيانات قبل تنفيذ COMMIT، أما الأخطاء العادية فلا تعني بالضرورة التراجع عن جميع العمليات المنفذة.

Auto Commit

- بعض أنظمة قواعد المعطيات تدعم خاصية Auto Commit، وهي خاصية تجعل النظام ينفذ COMMIT تلقائياً بعد كل تعليمة.
- وتكون قيمة المتغير: `auto_commit = true` أي يتم تثبيت كل عملية مباشرة بعد تنفيذها.
- أما إذا كانت: `auto_commit = false` فيمكن تنفيذ عدة تعليمات ضمن Transaction واحدة ثم تثبيتها أو التراجع عنها دفعة واحدة.

ملاحظة حول Oracle و SQL Server

- في Microsoft SQL Server يكون الوضع الافتراضي لـ Auto Commit هو True غالباً، لذلك يتم تثبيت كل تعليمة مباشرة.
- أما في Oracle فالوضع الافتراضي لا يقوم بالتثبيت إلا عند تنفيذ COMMIT .

- تعليمات DDL تقوم بالتثبيت تلقائياً
- تعليمات DDL (Data Definition Language) هي التعليمات الخاصة ببنية قاعدة البيانات، مثل: (CREATE TABLE - ALTER TABLE - ...)
- وفي معظم أنظمة قواعد المعطيات تؤدي هذه التعليمات إلى تنفيذ COMMIT

أنماط معطيات أخرى

تدعم SQL بالإضافة إلى الأنواع الأساسية (مثل INT و VARCHAR) مجموعة من الأنواع الخاصة للتعامل مع التاريخ والوقت والبيانات كبيرة الحجم.

1. DATE

يمثل تاريخاً فقط، ويتضمن: (السنة (Year) - الشهر (Month) - اليوم (Day))

مثال: '2005-07-27'

2. TIME

يمثل الوقت فقط، ويتضمن: (الساعة (Hours) - الدقيقة (Minutes) - الثانية (Seconds)) ، وقد يحتوي أيضاً على أجزاء من الثانية.

أمثلة: '09:00:30' - '09:00:30.75'

3. TIMESTAMP

يجمع بين التاريخ والوقت معاً في حقل واحد.

مثال: '2005-07-27 09:00:30.75' ، ويستخدم لتسجيل وقت حدوث حدث معين بدقة.

4. INTERVAL

يمثل فترة زمنية (مدة) وليس تاريخاً محدداً، ويستخدم غالباً كنتاج لطرح تاريخين أو وقتين من بعضهما. فينتج فرق زمني قد يكون بعدد الأيام أو الساعات أو الدقائق أو الثواني، حسب نوع البيانات المستخدمة.

5. BLOB (Binary Large Object)

يستخدم لتخزين البيانات الثنائية كبيرة الحجم، مثل: (الصور ، الملفات الصوتية، الفيديوهات).

6. CLOB (Character Large Object)

يستخدم لتخزين النصوص كبيرة الحجم، مثل:

(الملفات النصية الطويلة / المذكرات (Memo) - المستندات النصية الكبيرة text book)

ملاحظة

إذا احتوى الجدول على عمود من النوع BLOB أو CLOB، فلا يتم استخدام:

SELECT *

لأن هذه الحقول تحتاج إلى أدوات خاصة لعرض محتوياتها وإدخالها، وغالباً يتم اختيار الأعمدة المطلوبة فقط دون جلب البيانات الكبيرة تلقائياً.

الشروط الإضافية (CHECK)

- يُستخدم القيد CHECK لفرض شروط إضافية على قيم أحد الأعمدة أو مجموعة من الأعمدة، بحيث لا يُسمح بإدخال أو تعديل أي قيمة تخالف هذه الشروط.
- يتم التحقق من شرط CHECK عند تنفيذ عمليات INSERT أو UPDATE، فإذا لم تحقق البيانات الشرط المحدد يتم رفض العملية وعدم تنفيذها.
- وكلما كان شرط CHECK أكثر تعقيداً (مثل احتوائه على استعلامات فرعية أو عمليات كثيرة)، ازدادت الكلفة الزمنية لعمليات الإدخال والتحديث، لأن النظام يجب أن يتحقق من الشرط في كل مرة يتم فيها تعديل البيانات.

مثال :

```
create table section (
  course_id varchar (8),
  sec_id varchar (8),
  semester varchar (6),
  year numeric (4,0) default 2011,
  building varchar (15),
  room_number varchar (7),
  time_slot_id varchar (4),
  primary key (course_id, sec_id, semester, year),
  check (semester in ('Fall', 'Winter', 'Spring', 'Summer')),
  check (time_slot_id in (select time_slot_id from time_slot));
```

شرح التحقق الأولى

check (semester in ('Fall', 'Winter', 'Spring', 'Summer'))

- يفرض هذا الشرط أن تكون قيمة الحقل semester واحدة فقط من القيم الأربع المحددة، ولا يسمح بإدخال أي قيمة أخرى.

شرح التحقق الثانية

check (time_slot_id in (select time_slot_id from time_slot))

- يفرض هذا الشرط أن تكون قيمة time_slot_id موجودة مسبقاً ضمن جدول time_slot، أي لا يمكن إدخال قيمة غير موجودة في هذا الجدول.
- ويؤدي هذا الشرط وظيفة مشابهة للمفتاح الخارجي (Foreign Key) من حيث التحقق من وجود القيمة في جدول آخر، إلا أنه نُفذ هنا باستخدام CHECK مع استعلام فرعي.

العمليات الناتجة عن المفاتيح الخارجية

- عند وجود Foreign Key يربط جدولاً بجدول آخر، قد تحدث مشكلة عند حذف أو تعديل سجل من الجدول الأساسي بينما توجد سجلات مرتبطة به في الجدول التابع.

مثال :

```
create table course (
...
dept_name varchar(20),
foreign key (dept_name) references department
on delete cascade
on update cascade,
...
)
```

لذلك تسمح SQL بتحديد السلوك المطلوب باستخدام الخيارات التالية:

1. RESTRICT (القيمة الافتراضية)

- وهو الخيار الافتراضي في معظم أنظمة قواعد المعطيات.
- إذا كانت هناك سجلات مرتبطة بالمفتاح الخارجي، فإن النظام يمنع عملية الحذف أو التعديل حتى يتم حذف أو تعديل السجلات التابعة أولاً.

مثال:

إذا كان قسم Computer Science يحتوي على مواد دراسية، فلن يسمح بحذف هذا القسم طالما ما زالت هذه المواد مرتبطة به.

2. CASCADE

- عند حذف السجل الأساسي، تنتقل العملية تلقائياً إلى جميع السجلات التابعة له.
- ON DELETE CASCADE : يحذف جميع السجلات المرتبطة تلقائياً.
- ON UPDATE CASCADE : يحدّث قيم المفتاح الخارجي في جميع السجلات التابعة تلقائياً.

مثال:

- إذا حذف قسم Biology، فسيتم حذف جميع المواد التابعة له تلقائياً.
- وإذا تغير اسم القسم من Biology إلى Bio، فسيتم تحديث قيمة dept_name في جميع المواد التابعة تلقائياً.

3. SET NULL (NULLIFY)

- عند حذف أو تعديل السجل الأساسي، يتم تعيين قيمة المفتاح الخارجي في السجلات التابعة إلى NULL بدلاً من حذفها.
- أي تبقى السجلات موجودة، ولكنها تصبح غير مرتبطة بأي سجل في الجدول الأساسي.

مثال:

- إذا حذف قسم Physics، فستبقى المواد موجودة، لكن تصبح قيمة dept_name = NULL.
- ويشترط أن يسمح العمود بقيمة NULL.

4. SET DEFAULT

- عند حذف أو تعديل السجل الأساسي، يتم تعيين قيمة المفتاح الخارجي إلى القيمة الافتراضية (DEFAULT) المحددة مسبقاً للعمود.

مثال:

- إذا كانت القيمة الافتراضية لـ dept_name هي "General"، ثم حذف قسم Physics، فستصبح جميع المواد التابعة لهذا القسم مرتبطة بالقسم General بدلاً من القسم المحذوف.

ملاحظة مهمة

الخيارات الأربعة السابقة (RESTRICT – CASCADE – SET NULL – SET DEFAULT)

Delete + update

يمكن استخدامها مع كل من:



3 more pages
are still waiting
for u

إدارة الصلاحيات (Authorization)

- تستخدم إدارة الصلاحيات للتحكم في العمليات التي يستطيع كل مستخدم تنفيذها داخل قاعدة المعطيات، سواء على مستوى البيانات أو على مستوى بنية قاعدة البيانات.

صلاحيات على مستوى البيانات (Data Level) :

SELECT - INSERT - UPDATE - DELETE

صلاحيات على مستوى البنية (Schema Level) :

RESOURCES - ALTER - DROP - INDEX - REFERENCES

وبغیرها من الصلاحيات الخاصة بإدارة بنية قاعدة البيانات.

منح الصلاحيات (GRANT)

تستخدم التعليمة:

```
grant <privilege list>
on <relation name or view name>
to <user list>;
```

لمنح صلاحية أو أكثر لمستخدم واحد أو عدة مستخدمين على جدول أو رؤية معينة.

ملاحظة:

- يمكن أن تحتوي Privilege List على صلاحية واحدة أو عدة صلاحيات معاً.
- ويمكن أن تحتوي User List على مستخدم واحد أو عدة مستخدمين.
- أما الجزء: <relation name or view name> فيشير إلى جدول واحد فقط أو View واحدة فقط في كل تعليمة.



Keep IT up !!!

سحب الصلاحيات (REVOKE)

تستخدم التعليمة.

```
revoke <privilege list>
on <relation name or view name>
from <user list>;
```

ل سحب صلاحية أو أكثر من مستخدم أو عدة مستخدمين.

كما يمكن استخدام:

- **ALL** : لسحب جميع الصلاحيات.
- **PUBLIC** : للإشارة إلى جميع مستخدمي قاعدة البيانات.

يمكن منح الصلاحية مع السماح للمستخدم بإعطائها لمستخدمين آخرين باستخدام:

```
grant ...
with grant option;
```

أي أن المستخدم لا يملك الصلاحية فقط، بل يستطيع أيضاً منحها لغيره.

ملاحظة:

- إذا مُنحت الصلاحية باستخدام WITH GRANT OPTION، ثم قمنا بسحبها من المستخدم بواسطة: REVOKE فإن الصلاحية تُسحب منه فقط، بينما تبقى لدى المستخدمين الذين منحهم إياها سابقاً.
- أما إذا استخدمنا: REVOKE ... CASCADE فسيتم سحب الصلاحية من هذا المستخدم، وكذلك من جميع المستخدمين الذين حصلوا عليها عن طريقه بشكل متسلسل.

الأدوار (Roles)

- تُستخدم Roles لتسهيل إدارة الصلاحيات داخل قاعدة البيانات.
- وفكرتها هي تجميع مجموعة من الصلاحيات أو تجميع مجموعة من المستخدمين تحت اسم واحد، مما يسهل عملية الإدارة بدلاً من منح الصلاحيات لكل مستخدم بشكل منفصل.

مثال

```
create role instructor;
grant select on takes to instructor;
create role student;
grant instructor to Amin;
create role dean;
grant instructor to dean;
grant dean to Soshi;
```

الشرح :

- في هذا المثال تم إنشاء Role باسم instructor ومنحه صلاحية SELECT على جدول takes.
- ثم أعطي هذا الدور للمستخدم Amin فأصبح يملك هذه الصلاحية.
- بعد ذلك تم إنشاء Role آخر باسم dean، ومنحه دور instructor، أي أنه ورث جميع صلاحياته، ثم أعطي دور dean للمستخدم Soshi، فأصبح يمتلك صلاحيات dean التي تضم جميع صلاحيات instructor.
- وتوضح هذه الفكرة إمكانية إسناد Role إلى Role آخر لتسهيل إدارة الصلاحيات وإعادة استخدامها.

انتهى

